



Functions in C++

• function prototype

1. function declaration

type function-name (argument-list);

- * In a function declaration, the names of the arguments are dummy variable and therefore they are optional.

2. function definition

type function-name (argument-list);

- * In a function definition, the names are required.

3. function call

Important

- If we defined func() before function call then there is no need of function declaration.
- But if we define a function after function call then function declaration is must.



```

• #include <iostream.h>
  int sum (int, int);
  void main()
  {
    int a = 5, b = 6;
    int s = sum(a, b); → a and b are actual arguments
    cout << "sum = " << s;
  }

  int sum(int x, int y) → x and y are formal arguments
  {
    return (x+y);
  }

```

Formal arguments can be of three types :-

- Ordinary variables
- Pointer variables
- Reference variables

Call by Value :

```
#include <iostream>
int sum(int, int);
void main()
{
    int a = 5, b = 6;
    int s = sum(a, b);
    cout << "sum is " << s;
}
```

int sum(int x, int y) : When formal arguments are ordinary variables, it is function call by value
return(x+y);

Call by address :

```
#include <iostream>
int sum(int*, int*);
void main()
{
    int a = 5, b = 6;
    int s = sum(&a, &b);
    cout << "sum is " << s;
}
```

int sum(int* p, int* q) : When formal arguments are pointer variables, it is function call by address
return(*p + *q);



Call by reference:

```
#include <iostream>
```

```
int sum (int &, int &);
```

```
void main()
```

```
{
```

```
    int a = 5, b = 6;
```

```
    int s = sum(a, b);
```

```
    cout << "sum is " << s;
```

```
}
```

```
int sum (int &x, int &y) :
```

When formal arguments are reference variables, it is function call by reference

```
{
```

```
    return(x+y);
```

```
}
```